

Objetivo General:

El propósito de esta actividad es que los estudiantes construyan un sistema completo de flujo de datos, desde la lectura de un dataset hasta su visualización en una aplicación web pública.

La idea es simular cómo funciona un Data Warehouse moderno:

- Recolectar datos desde diferentes fuentes.
- Transmitirlos por la red usando protocolos de comunicación.
- Almacenarlos centralizadamente en una base de datos en la nube.
- Exponerlos y visualizarlos en un tablero accesible desde cualquier navegador.

Estructura general del sistema

El sistema que se construirá estará dividido en tres fases principales:

- Ingesta de datos (entrada de información).
- Procesamiento y almacenamiento.
- Exposición y visualización.

Cada fase representa un componente fundamental dentro de arquitecturas modernas de datos.

Fase 1: Ingesta de Datos

- Paso 1: Descarga del Dataset
 - El estudiante deberá descargar un conjunto de datos en formato CSV desde la plataforma Kaggle:
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
 - Este dataset contiene información real de transacciones comerciales, organizada en múltiples tablas, lo cual permite simular un entorno profesional.
 - Estructura del dataset
 - Algunos archivos relevantes incluyen:
 - Órdenes: identificadores, estados y fechas.
 - Productos: categorías y características.
 - Items de orden: precios y costos de envío.
 - Geolocalización: coordenadas geográficas.
 - **Importancia**
 - El uso de múltiples tablas permite practicar:
 - Modelado de datos.
 - Uso de consultas SQL con operaciones JOIN.
 - Simulación de esquemas tipo estrella o relacionales.
- Paso 2: Creación de Agentes Clientes
- Los agentes son programas que simulan sistemas reales que envían información a través de la red.
- Se deben implementar dos tipos:
 - a) Agente Transaccional (TCP)
 - Función: Simular sistemas críticos (por ejemplo, sistemas bancarios).
 - Características:
 - Utiliza el protocolo TCP (orientado a conexión y confiable).
 - Garantiza que los datos lleguen completos.

- Proceso:
 - Crear un socket tipo SOCK_STREAM.
 - Establecer conexión con el servidor en 127.0.0.1:12000.
 - Leer el archivo CSV línea por línea.
 - Enviar cada registro codificado en UTF-8.
 - Introducir pausas de 1 segundo para simular tráfico real.
- b) Agente de Telemetría (UDP)
 - Función: Simular sensores en tiempo real (temperatura, velocidad, etc.).
 - Características:
 - Utiliza UDP (rápido pero no garantiza entrega).
 - Adecuado para datos de alta frecuencia.
 - Proceso:
 - Crear un socket tipo SOCK_DGRAM.
 - Generar datos aleatorios.
 - Enviar los datos al servidor (127.0.0.1:12001).
 - Realizar envíos cada 0.5 segundos.

Fase 2: Recepción y Almacenamiento

- Paso 1: Desarrollo del servidor concurrente
 - El servidor será el componente central del sistema.
 - Funciones principales:
 - Recibir datos de múltiples clientes simultáneamente.
 - Gestionar conexiones TCP y UDP.
 - Evitar bloqueos mediante concurrencia
 - Implementación:
 - Crear un socket TCP en el puerto 12000.
 - Crear un socket UDP en el puerto 12001.
 - Utilizar hilos (threading) para manejar múltiples clientes.
 - Concepto clave: Concurrencia: Permite atender múltiples solicitudes al mismo tiempo, mejorando el rendimiento del sistema.
- Paso 2: Almacenamiento en la base de datos
 - Se utilizará una base de datos PostgreSQL en Supabase.
 - Proceso:
 - Configurar la conexión a la base de datos.
 - Recibir datos desde el servidor.
 - Decodificar la información.
 - Insertar los registros en una tabla central.
 - Interpretación:
 - origen: indica si los datos provienen de TCP o UDP.
 - contenido: almacena la información recibida.
 - fecha: registra el momento de inserción.

Fase 3: Exposición y Visualización

- Paso 1: Desarrollo de una API
 - Se debe implementar un servicio web utilizando:
 - Flask o FastAPI

- Objetivo: Permitir el acceso a los datos almacenados mediante solicitudes HTTP.
- Requerimiento:
 - Crear un endpoint: /api/datos
 - Realizar consultas SQL a la base de datos.
 - Devolver los resultados en formato JSON.
- Paso 2: Desarrollo del frontend
 - Se construirá una interfaz web básica.
 - Tecnologías: Reactjs
 - Biblioteca de gráficos ([Chart.js](#))
 - Funcionalidades:
 - Consumir la API mediante fetch().
 - Mostrar los datos en:
 - Tablas dinámicas.
 - Gráficas (por ejemplo: ventas por fecha).